GOVERN.AI

ANTHERA Systems

AUDIT TECNICO

VERSION
v3.1 - Sovereign Mode

DATE
21 Maggio 2026

AUTHOR
ANTHERA Systems

CLASSIFICATION
Confidential

European data deserves European AI.

Angelo Anglani - Founder & Lead Engineer
angelo.anglani94@gmail.com | +39 342 754 8655
linkedin.com/in/angelo-anglani

Audit tecnico interno della piattaforma GOVERN.AI. Aggiornato a valle del rilascio Sovereign Mode (v3.1).

Data: 21 Maggio 2026 (aggiornato post Sovereign Mode v3.1)

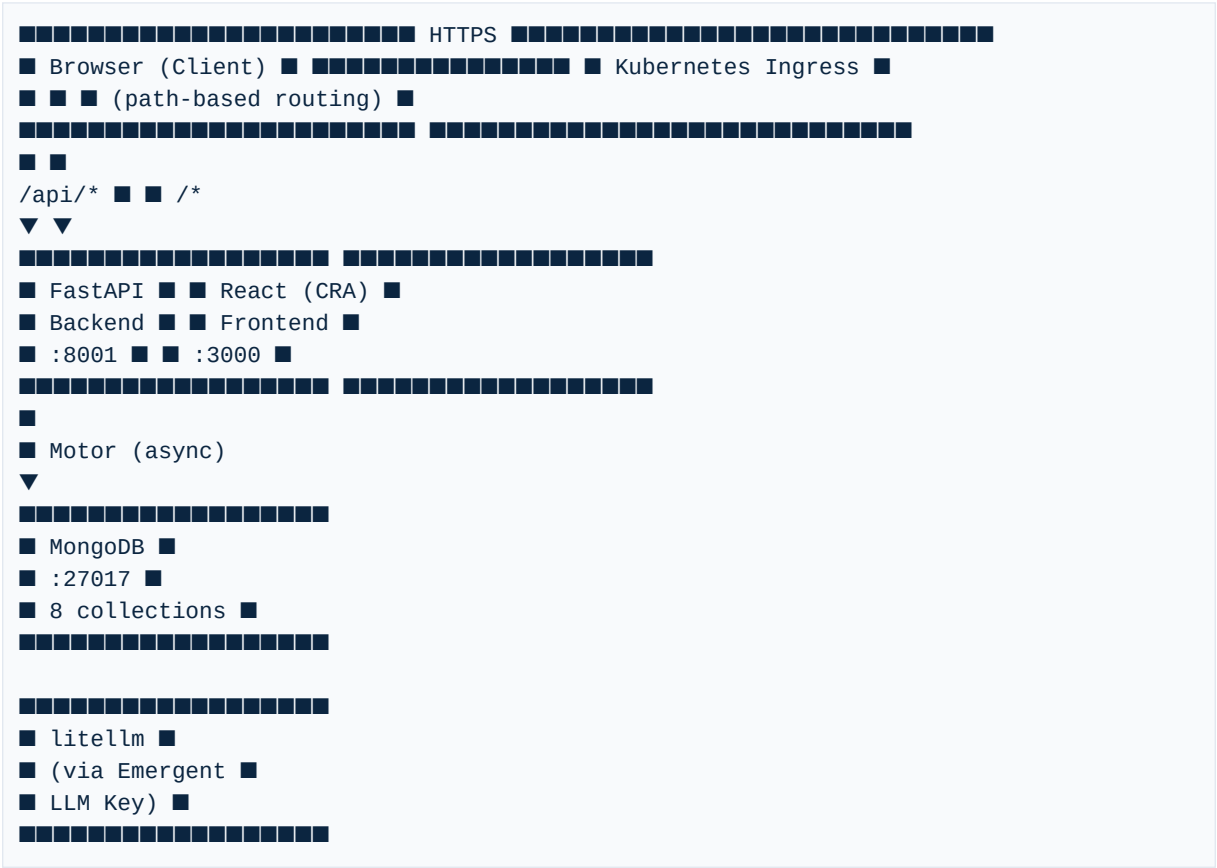
Versione codebase: v3.1 — Sovereign Mode

Autore: Audit automatico

---

1. PANORAMICA ARCHITETTURA ATTUALE

1.1 Schema a blocchi



1.2 Stack tecnologico

Componente	Tecnologia	Versione
Backend framework	FastAPI	0.110.1
Backend runtime	Python / Uvicorn	3.x / 0.25.0
Frontend framework	React	19.0.0
Frontend build tool	Create React App + CRACO	CRA 5.0.1, CRACO 7.1.0
Component library	Shadcn/UI (Radix primitives)	New York style
CSS framework	Tailwind CSS	3.4.17
Database	MongoDB (Motor async driver)	pymongo 4.5.0 / motor 3.3.1
LLM integration	litellm (configurabile via `LLM_MODEL`)	1.80.0
Validazione dati	Pydantic V2	2.12.5
Charts	Recharts	3.6.0
PDF Export	ReportLab	4.1.0
Rate Limiting	SlowAPI	—

1.3 Tipo di architettura

Architettura modulare a due tier con separazione frontend/backend:

- Backend: `server.py` (orchestratore) + 10 file di route modulari in `routes/` + `services/compliance_engine.py` + `models.py` + `database.py` + `seed.py` + `exporters.py`
- Frontend: SPA con routing lato client, 12 pagine, componente CRUD generico `CrudPage.js`
- Database: singola istanza MongoDB, 8 collections, 15+ indici
- Autenticazione: JWT con RBAC (4 ruoli)
- LLM: litellm (provider-agnostico)

2. STRUTTURA DEL CODICE

2.1 Directory tree (file rilevanti)

```
/app/
■■■ backend/
■ ■■■ server.py # FastAPI app + middleware + 10 router (~100 righe)
■ ■■■ models.py # 15 modelli Pydantic + 10 Enum (~250 righe)
■ ■■■ database.py # Connessione MongoDB + indici
■ ■■■ seed.py # Dati seed enterprise (~986 righe)
■ ■■■ exporters.py # PDF/CSV generation con ReportLab
■ ■■■ rate_limiter.py # Istanza condivisa SlowAPI
■ ■■■ services/
■ ■ ■■■ compliance_engine.py # Compliance Intelligence Engine (scoring)
■ ■■■ routes/
■ ■ ■■■ auth.py # Login, register, JWT, RBAC
■ ■ ■■■ agents.py # CRUD agenti AI
■ ■ ■■■ policies.py # CRUD policy
■ ■ ■■■ audit.py # Audit trail + export PDF/CSV
■ ■ ■■■ compliance.py # Standard compliance + export PDF
■ ■ ■■■ dashboard.py # Stats dashboard + KPI
■ ■ ■■■ chat.py # ARIA AI assistant (SSE streaming)
■ ■ ■■■ sox_wizard.py # SOX Section 404 Wizard + Readiness Score
■ ■ ■■■ policy_engine.py # Policy Conflict Detection + Guidance Engine
■ ■ ■■■ score.py # Compliance Intelligence Engine API (6 endpoint)
■ ■■■ tests/
■ ■■■ test_api.py # Suite test API (50 test completi)
■■■ frontend/
■ ■■■ src/
■ ■■■ contexts/
■ ■ ■■■ AuthContext.js
■ ■ ■■■ LanguageContext.js
■ ■■■ pages/ (12 pagine)
■ ■ ■■■ LandingPage.js
■ ■ ■■■ LoginPage.js
■ ■ ■■■ DashboardLayout.js
■ ■ ■■■ OverviewPage.js
■ ■ ■■■ AgentsPage.js
■ ■ ■■■ PoliciesPage.js
■ ■ ■■■ AuditPage.js
■ ■ ■■■ CompliancePage.js
■ ■ ■■■ AssistantPage.js
■ ■ ■■■ SoxWizardPage.js
■ ■ ■■■ PolicyEnginePage.js
■ ■ ■■■ IntelligenceCenterPage.js # Intelligence Center (v3.0)
■ ■■■ components/
■ ■ ■■■ CrudPage.js, Logo.js, EmptyState.js, SkeletonLoader.js
■ ■ ■■■ ui/ (~39 componenti Shadcn)
■ ■■■ locales/
■ ■■■ en.json (~185 chiavi)
■ ■■■ it.json (~185 chiavi)
■■■ docker-compose.yml
■■■ .github/workflows/ci.yml
■■■ README.md
```

2.2 Analisi file principali

File	Righe	Responsabilita
`server.py`	~100	App FastAPI, middleware sicurezza, include 10 router
`models.py`	~250	15 modelli Pydantic + 10 Enum
`seed.py`	~986	Dati seed enterprise banking
`services/compliance_engine.py`	~350	Motore di scoring deterministico
`routes/score.py`	~130	6 endpoint API per il motore di scoring
`routes/policy_engine.py`	~350	Conflict detection + guidance + resolve
`IntelligenceCenterPage.js`	~300	Intelligence Center premium page
`PolicyEnginePage.js`	~300	Policy Engine con guidance e resolve

3. DATABASE & MODELLO DATI

3.1 Elenco collections (8)

Collection	Documenti	Campi principali
`users`	1+	id, username, email, password_hash, role
`agents`	14	id, name, model_type, risk_level, status, allowed_actions, owner
`policies`	20+	id, name, agent_id, rule_type, conditions, actions, severity, regulation, enforcement
`audit_logs`	150+	id, timestamp, agent_name, action, resource, outcome, risk_level, user
`compliance_standards`	8	id, name, code, progress, requirements_total/met
`chat_messages`	variabile	session_id, role, content, timestamp
`sox_controls`	20	id, domain, control_id, title, status, evidence, risk_level
`score_history`	variabile	timestamp, overall_score, agent_snapshots, standard_snapshots

3.2 Modelli Pydantic (15 totali)

Modello	Scopo
`UserCreate`, `UserLogin`, `UserOut`	Autenticazione utenti
`AgentCreate`, `Agent`	Registro agenti AI
`PolicyCreate`, `Policy`	Motore policy
`AuditLog`	Traccia audit
`ComplianceStandard`	Standard normativi
`SoxControl`, `ControlStatus`	SOX Section 404
`PolicyConflict`, `ConflictType`, `ConflictSeverity`	Policy Conflict Engine
`ConflictResolution`	Risoluzione documentata conflitti
`ChatRequest`	Messaggi chat

3.3 Enum (10)

`RiskLevel`, `AgentStatus`, `PolicySeverity`, `PolicyEnforcement`, `RuleType`, `AuditOutcome`, `DataClassification`, `UserRole`, `ControlStatus`, `ConflictType`, `ConflictSeverity`

4. SICUREZZA

Meccanismo	Stato
JWT Auth (HS256, 8h)	Implementato
RBAC 4 ruoli (admin > dpo > auditor > viewer)	Implementato
Rate Limiting (SlowAPI)	Implementato
CORS restrittivo (da env)	Implementato
Security Headers (5)	Implementato
Sanitizzazione regex	Implementato
Enum Pydantic (10)	Implementato
LLM error masking	Implementato
bcrypt password hashing	Implementato

5. TESTING & QUALITA

5.1 Test automatici: 50/50 passati

Area	Test	Stato
Auth	login, register, token	3/3
Agents	CRUD completo	4/4
Policies	CRUD completo	4/4
Audit Trail	query, filtri, export PDF/CSV	4/4
Compliance	list, export PDF, 8 standards	3/3
Dashboard	stats	1/1
Chat	ARIA query	1/1
SOX Wizard	controls, patch, report JSON/PDF	4/4
Readiness Score	score calculation	1/1
Policy Engine	conflicts, resolution, gaps, scan history	4/4
Policy Guidance	guidance endpoint, mandatory notes, short notes 422, audit log	5/5
Compliance Intelligence Engine	overview, explainability, agents, single, 404, standards, history, insights, conflict integration, bands, remediations	11/11
Standards validation	7+8 standards	2/2
Misc	root, RBAC	3/3

5.2 CI/CD: GitHub Actions (4 job)

`backend-tests` (50 pytest), `frontend-build`, `security-scan`, `docker-build`

5.3 Testing agent: 9 iterazioni, tutte passate 100%

6. COMPLIANCE INTELLIGENCE ENGINE (v3.0) — Cuore Proprietario

6.1 Architettura

```
services/compliance_engine.py
■■■ score_agent() → Score 0-100 per agente
■■■ score_standard() → Score 0-100 per standard normativo
■■■ calculate_overview() → Score complessivo + explainability
■■■ compute_full_scores() → Orchestratore (fetch data → score → aggregate)
■■■ compute_and_snapshot()→ Calcola + salva snapshot per history
```

6.2 Formula di Scoring

Agent Score (0-100):

- Base da risk_level: critical=25, high=45, medium=65, low=82
- Policy coverage: +15 max (con policy) / -20 (senza policy = gap)
- Audit outcome: +/-15 (ratio allowed/total actions)
- Conflict penalty: critical=-18, high=-10, medium=-5, low=-2
- Status: active=0, suspended=-10, inactive=-20

Standard Score (0-100):

- Base: progress %
- Requirements bonus: delta req_ratio vs progress
- Policy coverage: +12 max / -8 (senza policy)
- Conflict penalty per regulation

Overall: standards 55% + agents 45% - penalita conflitti critici (max -15)

6.3 Explainability Layer

Ogni score include:

- `explanation_summary` — riassunto leggibile
- `why_this_score` — formula applicata
- `strongest_positive_factor` / `strongest_negative_factor`
- `methodology_note` — "Scores are deterministic, no LLM involved"
- `score_factors` — conteggi (excellent, warning, critical, conflicts)
- `score_breakdown` — decomposizione numerica del punteggio

6.4 Score History & Momentum

- Snapshot salvati in collection `score_history`
- Campi: `previous_score`, `delta_score`, `trend_direction` (up/down/stable)
- Disponibile per agenti e standard

6.5 Integrazione Conflitti

- Conflitti critici non risolti → penalita score agente (-18 pts)
- Gap (agenti senza policy) → missing controls
- Overlap/redundancy → remediation suggestions
- Il Policy Conflict Engine alimenta direttamente il motore di scoring

---

7. FUNZIONALITA IMPLEMENTATE — STORICO COMPLETO

Step	Versione	Funzionalita
MVP v1.0	v1.0	Landing, Dashboard, CRUD, Audit Trail, 6 standard, ARIA, i18n
Step 1	v1.1	15 indici MongoDB, regex sanitization, CORS, Enum, debounce
Step 2A	v1.2	JWT + RBAC (4 ruoli), ARIA verticale, rate limiting
Step 2B	v1.3	Backend modulare (9 route), security headers, CrudPage
Step C1	v1.4	Dashboard Recharts (3 grafici), enterprise seed data
Step C2	v1.5	Export PDF/CSV (Audit + Compliance)
Step C3A	v1.6	Logo ufficiale, mobile sidebar responsive
Step C3B	v1.7	Docker + docker-compose, README professionale
Step CICD	v1.8	GitHub Actions CI (4 job paralleli)
Step FINAL	v1.9	Landing use cases, SSE streaming ARIA, empty states, titoli
Fix	v2.0	Audit chart, delete dialog, portabilita LLM (litellm)
Step E1	v2.1	SOX Foundation (standard + agente + 3 policy + audit cluster)
Step E2	v2.2	SOX 404 Wizard (20 controlli, 5 domini, report PDF)
Step E3	v2.3	D.Lgs. 262/2005 (8o standard) + Audit Readiness Score
Step E4	v2.4	Policy Conflict Detection Engine (4 regole, 3 endpoint, UI)
Step E5	v2.5	Policy Guidance Engine (guidance, impact, risoluzione documentata)
Step E6	v3.0	Compliance Intelligence Engine + Explainability + Intelligence Center + Score History + ARIA upgrade

8. DEBITO TECNICO RESIDUO

ID	Area	Problema	Priorita
TD-FE1	Frontend	Nessun test unitario frontend (Jest)	P2
TD-BE1	Backend	Query dashboard non aggregate in pipeline	P2
TD-BE2	Backend	Paginazione audit solo backend	P2
TD19	Database	Date come stringhe ISO	P3

9. DA COMPLETARE

- **Test unitari frontend** (Jest + Testing Library) — P2
- **Connettori enterprise** (IAM, SIEM, ServiceNow) — P2
- **Multi-tenancy** — P2
- **D.Lgs. 262 Wizard** (workflow dedicato) — P2
- **WebSocket real-time monitoring** — P2

SOVEREIGN MODE — APERTUS INTEGRATION (v3.1)

Panoramica

GOVERN.AI v3.1 introduce supporto nativo per Apertus (Swiss AI Initiative — ETH Zurich + EPFL + CSCS) come provider LLM sovrano via litellm.

File modificati/aggiunti

File	Tipo
backend/settings.py	NUOVO — singleton config LLM
backend/routes/chat.py	MODIFICATO — sovereign routing + fallback GPT-4o
backend/routes/ai_settings.py	NUOVO — endpoint REST admin-only
backend/server.py	MODIFICATO — registrazione router
frontend/src/pages/SettingsPage.js	NUOVO — UI toggle
frontend/src/components/AIModeBadge.js	NUOVO — badge sidebar
.env.example	MODIFICATO — +4 variabili Apertus

Architettura dual-provider

litellm routing:

- LLM_SOVEREIGN_ENABLED=true + PUBLICAI_API_KEY valorizzata → Apertus-70B
- Altrimenti → GPT-4o (default)
- Errore Apertus → fallback automatico GPT-4o (try/except chat.py)

Nuove variabili ambiente

Variabile	Default
LLM_SOVEREIGN_ENABLED	false
LLM_SOVEREIGN_MODEL	publicai/swiss-ai/apertus-70b-instruct
LLM_SOVEREIGN_BASE_URL	https://platform.publicai.co/v1
PUBLICAI_API_KEY	your_key_here

Endpoint REST

GET /api/settings/ai-mode → stato corrente (viewer+) POST /api/settings/ai-mode → toggle (admin only)

Validazione

50/50 pytest PASSED — zero regressioni Fallback automatico GPT-4o confermato nei log

Fine audit tecnico. Ultimo aggiornamento: 21 Maggio 2026 (v3.1 — Sovereign Mode).